



Databases & SQL

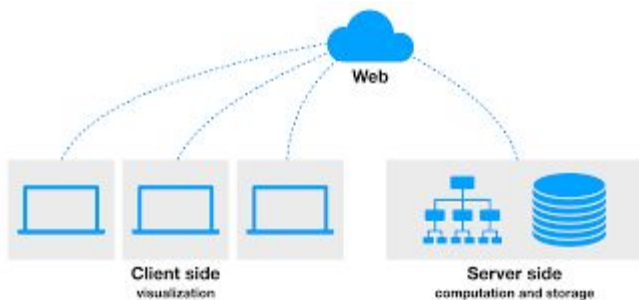


# Learning Objectives

- Explain that there are relational and non-relational databases and that postgres is a relational database
- Explain that SQL is a programming language that we use to query a set of databases
- Explain that a database is a specialised file system
- Explain that databases are usually managed by an application running on a server
- Explain that databases are collections of tables and can be created and dropped
- Create a database
- Use a tool to get visibility on the state of the database
- Use SQL to create and drop a table in the database
- Explain that a table has rows and columns also known as records and fields
- Use SQL to insert and select record in a table
- Explain that each record should have an identifier known as a primary key



# A Single Source of Truth



When we think about the web, it's important to remember there might be multiple users (clients) interacting with a website or app who expect to access their data later in time.

To make sure that everyone has access to relevant information we need to **manage and store that information**

We use a **database server** to manage accessing and storing that information – the data in the database is usually the source of truth for all of our users.

We use an **application server** to manage general access to that information by enforcing business rules and constraints via an API

**Q:** If we didn't have a **single source of truth** on an ecommerce site, what problems could occur?



# Let's Diagram!

**How do our Frontend, API, and  
Database work together?**



# What Database?



**Relational**



**Non-Relational**

These are just some of the available databases in each category...there are many, *many* more!

There are many different types of Database, split into two broad categories: **relational** and **non-relational**.

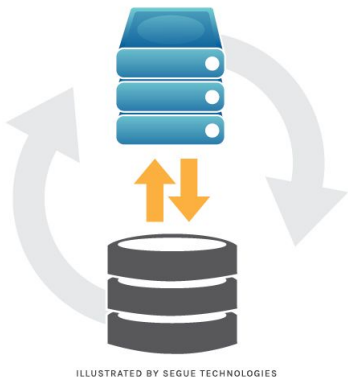
Relational databases have been around forever; they are solid, reliable and general purpose.

Non-relational databases, sometimes referred to as “NoSQL databases”, are more recent and tend to be engineered for specific use-cases.

We'll be using **Postgres**, a relational database



# The Server & Database Relationship



ILLUSTRATED BY SEGUE TECHNOLOGIES

Just like in any relationship **communication is important**

If a server wants something from the database it sends a **query**

If the database has something for the server it sends back **data**

To communicate they use a **query language**. Relational databases use a query language called **SQL** (**S**tructured **Q**uery **L**anguage).



# How to query a Relational DB; Queries

**Create**

**INSERT**

**Read**

**SELECT**

**Update**

**UPDATE**

**Delete**

**DELETE**



# How to setup a SQL database



There are two ways to setup a postgres database

- In a **local environment** via your computer
- In the **cloud** via a service

We will be using a database in the **cloud**

**RESOURCE:** [Elephant SQL Documentation: Getting Started](#)





# How to setup a SQL database



There are two ways to setup a postgres database

- In a **local environment** via your computer
- In the **cloud** via a service

We will be using a database in the **cloud**

RESOURCE: [Neon Introduction](#)



# How does the Database represent data?

```
"products": [  
  {  
    "id": 2,  
    "name": "Banana",  
    "price": 0.35,  
    "discount": false  
  },  
  {...}  
]  
  
"users": [  
  {  
    "id": 1,  
    "name": "Nico"  
  },  
  {...}  
]  
  
"orders": [  
  {  
    "id": 1,  
    "productId": 2,  
    "userId": 1,  
    "quantity": 8  
  },  
  {...}  
]
```

We have seen **JSON**

But SQL databases don't work with JSON... they work with **tables**, **columns** and **rows**

**Q:** What do you think are the tables? What do you think are the columns? What do you think are the rows?



# Tables, Columns and Rows

**products**

| id | name   | price | discount |
|----|--------|-------|----------|
| 1  | Banana | 0.35  | false    |
| 2  | Apple  | 0.2   | true     |

**users**

| id | name |
|----|------|
| 1  | Nico |
| 2  | Fred |
| 3  | Bob  |

**orders**

| id | productId | userId | quantity |
|----|-----------|--------|----------|
| 1  | 2         | 1      | 8        |
| 2  | 1         | 1      | 5        |

The **tables** are products, users and orders

The **columns** for products are *"id"*, *"name"*, *"price"* and *"discount"*. These are sometimes referred to as fields.

The **rows** are the actual data. These are sometimes referred to as records.



# How to query the DB; READ with SELECT

```
SELECT *  
FROM products;
```

**Q:** What do you think this query does?



# How to query the DB; READ with SELECT

```
SELECT *  
FROM products;
```

## products

| id | name   | price | discount |
|----|--------|-------|----------|
| 1  | Banana | 0.35  | false    |
| 2  | Apple  | 0.2   | true     |

It's going to give us rows of products **with every column** in the products table



# How to query the DB; READ with SELECT

```
SELECT name, price  
FROM products;
```

**Q:** What do you think this query does?



# How to query the DB; READ with SELECT

```
SELECT name, price  
FROM products;
```

## products

| name   | price |
|--------|-------|
| Banana | 0.35  |
| Apple  | 0.2   |

It's going to give us rows of products **with the name and price columns** in the products table



# How to query the DB; constraints

```
SELECT column, another_column, ...  
FROM mytable  
WHERE condition(s)  
ORDER BY column ASC/DESC  
LIMIT num_limit OFFSET num_offset;
```

When we read from the database we can use **constraints** to select certain rows that **meet the right conditions** using **WHERE**

And we can **manipulate the rows** using **ORDER BY** and **LIMIT**





# Time for some live-coding

Demonstrate in [Elephant SQL](#)

- **SELECT** queries



# Time for some live-coding

Demonstrate in [Neon](#)

- `SELECT` queries



# Time for some practice

Go to [SQL Bolt](#) and do the interactive exercises:

- From: "SQL Lesson 1: SELECT queries 101"
- To: "SQL Review: Simple SELECT Queries"



## How to query the DB; CREATE with INSERT

```
INSERT INTO mytable  
    (column_one, column_two, ...)  
VALUES  
    (value_for_column_one, value_for_column_two, ...)
```

We use **INSERT INTO** to create rows in a table, let's have a look at an insert for our products table



## How to query the DB; CREATE with INSERT

```
INSERT INTO products
    (name, price, discount)
VALUES
    ("orange", 0.60, false)
```

**Q:** Why are we not adding an id?



# How to query the DB; CREATE with INSERT

```
INSERT INTO products
    (name, price, discount)
VALUES
    ("orange", 0.60, false)
```

## products

| id | name   | price | discount |
|----|--------|-------|----------|
| 1  | Banana | 0.35  | false    |
| 2  | Apple  | 0.2   | true     |
| 3  | orange | 0.60  | false    |

Databases will auto-increment the ids and handle that step for us



# Time for some live-coding

Demonstrate in [Elephant SQL](#)

- INSERT queries



# Time for some live-coding

Demonstrate in [Neon](#)

- INSERT queries





# Time for some practice

Go to [SQL Bolt](#) and do the interactive exercise

- Lesson 13: Inserting rows

Link [here](#)



## But...where do tables come from?

Before we can INSERT and SELECT data from a table, we need to CREATE it in the database first. Here's the syntax:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(20) NOT NULL,  
  email VARCHAR(50) NOT NULL,  
  UNIQUE (email)  
);
```

We have a **CREATE TABLE** query for a **users** table

We have an **id**, **name** and email column

**Q:** What do you think **SERIAL** and **VARCHAR(20)** are for?

### REFERENCE

<https://www.postgresqltutorial.com/postgresql-create-table/>



# CREATE TABLE

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(20) NOT NULL,  
  email VARCHAR(50) NOT NULL,  
  UNIQUE (email)  
);
```

**SERIAL** and **VARCHAR()** are data types for each column

**SERIAL** auto-increments the **id** column

**VARCHAR(20)** accepts 20 characters in the **name** column

**VARCHAR(50)** accepts 50 characters in the **email** column

**Q:** What do you think **PRIMARY KEY**, **NOT NULL** and **UNIQUE** are for?

## REFERENCE:

<https://www.postgresqltutorial.com/postgresql-data-types/>  
<https://www.postgresql.org/docs/current/datatype.html>



# CREATE TABLE

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(20) NOT NULL,  
  email VARCHAR(50) NOT NULL,  
  UNIQUE (email)  
);
```

**PRIMARY KEY**, **NOT NULL** and **UNIQUE** are **constraints**

**NOT NULL** means when a new row is created there needs to be a value

**UNIQUE** means when a new row is created a row with the same value should not exist

And **PRIMARY KEY** is **NOT NULL** and **UNIQUE** combined – **every table must have a primary key** – it uniquely identifies each row in the database! It's **very common** to use SERIAL to create an automatically incrementing id.

**Q:** How would you describe this **schema** in plain english?



# CREATE TABLE

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(20) NOT NULL,  
  email VARCHAR(50) NOT NULL,  
  UNIQUE (email)  
);
```

“We have a user table in our database, each user will have an **id**, a **name and an email**.

The id is auto-incremented by the database so we don't need to worry about it... the name needs to be a string of 20 characters or less. The email needs to be a string of 50 characters or less and it is unique so there shouldn't be any duplicate emails in our database.”



# DROP TABLE

```
DROP TABLE users
```

When we want to delete a table or recreate it with different columns we can DROP it – this removes all of your data!

It is possible to ALTER tables too to add or remove columns or change columns. That can get more complex, but we'll cover it later in the course.



# TRUNCATE

```
TRUNCATE users
```

If we want to remove all the data from our table, but keep the table definition, we can use TRUNCATE.



## VIEWING TABLE METADATA

```
SELECT
```

```
    table_name,
```

```
    column_name,
```

```
    data_type
```

```
FROM
```

```
    information_schema.columns
```

```
WHERE
```

```
    table_name = 'products';
```

If we're working with a table and we're not sure what columns it has or what their data types are, we can use this query to find out.





# Time for some live-coding

Demonstrate in [Elephant SQL](#)

- CREATE TABLE
- DROP TABLE
- TRUNCATE



# Time for some live-coding

Demonstrate in [Neon](#)

- CREATE TABLE
- DROP TABLE
- TRUNCATE



# Time for some practice

- **Create your own account on Neon**
- **Create a users table**
- **Insert some rows and select them**
- **Truncate the users table**
- **Drop the users table**



# Database Concept; relationships

**products**

| id | name   | price | discount |
|----|--------|-------|----------|
| 1  | Banana | 0.35  | false    |
| 2  | Apple  | 0.2   | true     |

**users**

| id | name |
|----|------|
| 1  | Nico |
| 2  | Fred |
| 3  | Bob  |

**orders**

| id | productId | userId | quantity |
|----|-----------|--------|----------|
| 1  | 2         | 1      | 8        |
| 2  | 1         | 1      | 5        |

When we build databases we separate our data into different tables BUT **most of the time there will be some kind of relationship** between them

**Q:** Which tables share a relationship with each other?



# Database Concept; relationships

**products**

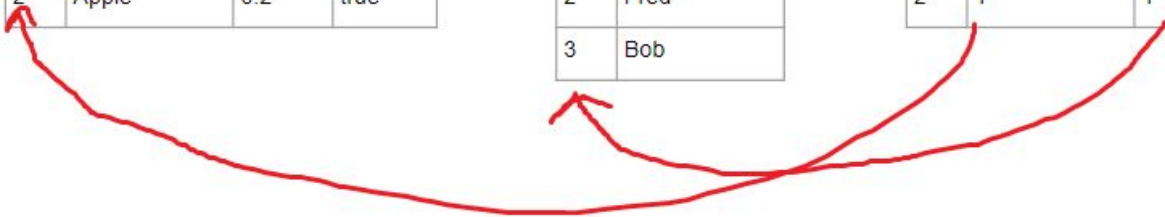
| id | name   | price | discount |
|----|--------|-------|----------|
| 1  | Banana | 0.35  | false    |
| 2  | Apple  | 0.2   | true     |

**users**

| id | name |
|----|------|
| 1  | Nico |
| 2  | Fred |
| 3  | Bob  |

**orders**

| id | productId | userId | quantity |
|----|-----------|--------|----------|
| 1  | 2         | 1      | 8        |
| 2  | 1         | 1      | 5        |



We use **foreign keys** (ids that connect a table to another table) and **JOINS** to create those relationships



# How to query the DB; JOINS

```
SELECT column, another_table_column, ...  
FROM mytable  
INNER JOIN another_table  
    ON mytable.id = another_table.id
```

This is just a quick introduction to the concept of **relationships** and **JOINS** we'll be implementing them in more detail later...

For now let's take a look at SQL Bolt and do some together!



# Time for some practice

Go to [SQL Bolt](#) and do the interactive exercise